CARLETON UNIVERSITY

SCHOOL OF
MATHEMATICS AND STATISTICS

HONOURS PROJECT

TITLE: Computation of the Smallest Eigenvalue for Dirichlet-Neumann Boundary Sturm-Liouville Problems

AUTHOR: Adam Miles

SUPERVISOR: Professor Angelo B. Mingarelli

DATE: April 30, 2026

Acknowledgments

I would like to thank Professor Angelo B. Mingarelli, my project supervisor. He sparked my interest in mathematics and gave me some interesting problems to solve for this project. Throughout my work, he has always been very kind, patient and helpful. Another person who I would like to thank is my second reader, Professor Emmanuel Lorin for taking the time to make suggestions for my project.

I am also very grateful to my family and friends for supporting me while I wrote this project. They always believed in me and encouraged me to continue to pursue my interests in mathematics.

Abstract

In this project, we have the objective of approximating the smallest eigenvalue for Sturm-Liouville problems with Dirichlet-Neumann boundary conditions. To do so, we use a Prüfer substitution based shooting method implemented in python. Some basic results regarding Sturm-Liouville theory and the Prüfer substitution that justify and explain the shooting method are presented. The method is also tested against some examples.

Contents

1 Introduction

We begin by introducing the **Sturm-Liouville equation**, a type of second order linear differential equation of the form.

$$[p(x)y']' + [\lambda w(x) - q(x)]y = 0 \tag{1.1}$$

Where p, q and w are real valued functions defined on a closed interval $[a, b] \subset \mathbb{R}$ and $\lambda \in \mathbb{C}$. We also impose some boundary conditions on the equation of the form

$$\alpha_0 y(a) + \beta_0 y'(a) = 0, \alpha_1 y(b) + \beta_1 y'(b) = 0 \tag{1.2}$$

The boundary conditions here are called **separated** since each only depends on one boundary point. Combining (1.1) and (1.2), we have a **Sturm-Liouville problem**. To solve a Sturm-Liouville problem, we must find a non-trivial function y and an associated value of λ satisfying (1.1) and (1.2). Such values of λ are called **eigenvalues** and its associated solution functions are called **eigenfunctions** (should the solutions exist). For this project, we assume that the Sturm-Liouville problems are **regular** which means that on the interval $[a, b]$, q and w are continuous and p is continuously differentiable with $p, w > 0$.

Often, it is not possible to find a solution to Sturm-Liouville problems analytically. For this reason, we require numerical algorithms to approximate eigenvalues and eigenfunctions. In this project, we use a shooting method

to approximate the smallest eigenvalue for certain Sturm-Liouville problems with Dirichlet-Neumann boundary conditions. In this project, we present some basic results for Sturm-Liouville problems, the Prüfer substitution, describe a shooting method, and provide an implementation and numerical experiments.

2 Basic Sturm-Liouville theory

Here we provide some basic theorems to show that the problem we want to solve theoretically has a solution. In particular, we want to show that the eigenvalues of (1.1, 1.2) are ordered and that there is a smallest eigenvalue. Here are the results.

Theorem 2.1. *The eigenvalues of the Sturm-Liouville problem (1.1, 1.2) are real numbers. [1]*

Theorem 2.2. *Each eigenvalue λ_n for the Sturm-Liouville Problem (1.1, 1.2) corresponds to exactly one linear independent eigenfunction. Additionally, the eigenvalues form a strictly increasing infinite sequence $(\lambda_n)_{n\in\mathbb{N}}$ where $\lim_{n\to\infty} \lambda_n = \infty$. [1] Furthermore, the eigenfunction y_n with the eigenvalue λ_n has n zeros. [2]*

From the theorem above, we conclude that there is indeed a smallest eigenvalue, λ_1 for a given Sturm-Liouville problem. In the next chapters, we develop some theory with the Prüfer substitution. This will lead into an algorithm to approximate λ_1.

3 The Prüfer Substitution

Consider the following second order differential equation

$$[P(x)y'(x)]' + Q(x)y(x) = 0 \tag{3.1}$$

where we assume that $P(x) > 0$ is continuously differentiable and $Q(x) > 0$ is continuous on $[a, b] \subseteq \mathbb{R}$. We can study properties of the solution function, y to (3.1) using a technique called the Prüfer substitution defined by the equations below:

$$P(x)y'(x) = r(x)\cos\theta(x) \tag{3.2a}$$

$$y(x) = r(x)\sin\theta(x) \tag{3.2b}$$

We call $r(x)$ the amplitude and $\theta(x)$ the phase variable. For that definition to make sense, we define r and θ.

$$r = \sqrt{y^2 + (Py')^2} \tag{3.3a}$$

$$\theta = \arctan(\frac{y}{Py'}) \tag{3.3b}$$

We now want to find a new system of differential equations for r and θ that does not involve y. To simplify later calculations, we first re-write (3.3a) and (3.3b) as

$$r^2 = y^2 + (Py')^2 \tag{3.4a}$$

$$\cot\theta = \frac{Py'}{y} \tag{3.4b}$$

Implicitly differentiating (3.4b) gives the equation

$$-\csc^2\theta\frac{d\theta}{dx} = \frac{y\cdot(Py')' - y'(Py')}{y^2} = \frac{(Py')'}{y} - \frac{P\cdot(y')^2}{y^2}$$

Equations (3.2a) and (3.4b) can be rearranged to obtain $Q(x) = -\frac{(Py')'}{y}$ and $\cot^2\theta = \frac{P^2\cdot(y')^2}{y^2}$. Now we can rewrite the equation above without y as

$$-\csc^2\theta\frac{d\theta}{dx} = -Q(x) - \frac{\cot^2\theta}{P(x)}$$

Hence, we now write the following first order differential equation for θ:

$$\frac{d\theta}{dx} = Q(x)\sin^2\theta + \frac{\cos^2\theta}{P(x)} \tag{3.5}$$

We follow a similar process to obtain an equation for r. Implicitly differentiating (3.4a) gives

$$2r\frac{dr}{dx} = 2y'y + 2(Py')(Py')'$$

By the definition of the Prüfer substitution and (3.1), we have

$$\begin{aligned} 2r\frac{dr}{dx} &= 2(r\cos\theta)(-Qy) + 2(r\sin\theta)y \\ &= -2Qr^2\cos\theta\sin\theta + \frac{2r^2\sin\theta\cos\theta}{P} \\ &= 2r^2\sin\theta\cos\theta\left(\frac{1}{P} - Q\right) \end{aligned}$$

Now, simplifying the expression yields

$$\frac{dr}{dx} = \frac{1}{2}r(x)\sin 2\theta\left[\frac{1}{P(x)} - Q(x)\right] \tag{3.6}$$

Together, (3.5) and (3.6) are called the **Prüfer system** associated with (3.1). We impose the following initial conditions for (3.1)

$$\begin{aligned} y(a) &= r(a)\sin[\theta(a)] \\ y'(a) &= \frac{r(a)\cos[\theta(a)]}{P(a)} \end{aligned}$$

We now state a theorem explaining how solving the Prüfer system and (3.1) are equivalent. [3]

Theorem 3.1. *Suppose that equations (3.1), (3.5) and (3.6) are valid. Then,*

1. *If θ is a solution to (3.5) and r is a solution to (3.6) then $y = r\sin\theta$ is a solution to (3.1) and $Py' = r\cos\theta$.*

2. *If y is a non-trivial solution to (3.1) then equations (3.5) and (3.6) have solutions θ and r such that $y = r\sin\theta$ and $Py' = r\cos\theta$.*

Now, we require that (3.5) and (3.6) have unique solutions for y to have a solution. We recall the existence and uniqueness theorem for first order ordinary differential equations so that we can determine if those equations have solutions.

Theorem 3.2. *Let D be an open and connected subset of $\mathbb{R}^2$ containing the point (t_0, z_0) and let $S \subset D$ be a rectangle*

$$S = \{(t, z) \in \mathbb{R}^2 : |t - t_0| \leq a, |z - z_0| \leq b\}$$

We also assume that $f : D \to \mathbb{R}$ is locally Lipschitz continuous with respect to z and is continuous as a two-variable function. That is, for each $(t, z_1), (t, z_2) \in S$, there is a Lipshitz constant $L > 0$ such that

$$|f(t, z_1) - f(t, z_2)| \leq L|z_1 - z_2|$$

Then, there is an open interval $(t_0 - h, t_0 + h) \subset [\alpha, \beta]$ where the initial value problem

$$z'(t) = f(t, z) \tag{3.8a}$$

$$z(t_0) = z_0 \tag{3.8b}$$

has a unique solution. [4]

Now we look at the two variable function

$$F(x,\theta) := Q(x)\sin^2\theta + \frac{\cos^2\theta}{P(x)} \tag{3.9}$$

Taking the partial derivative of F with respect to θ, we have

$$\begin{aligned}\frac{\partial F}{\partial \theta} &= 2Q(x)\sin\theta\cos\theta - 2\frac{\sin\theta\cos\theta}{P(x)} \\ &= Q(x)\sin 2\theta - \frac{\sin 2\theta}{P(x)}\end{aligned}$$

Hence by the triangle inequality,

$$\begin{aligned}\left|\frac{\partial F}{\partial \theta}\right| &\leq |Q(x)| + \frac{1}{|P(x)|} \\ &\leq \sup_{x\in[a,b]} |Q(x)| + \sup_{x\in[a,b]} \frac{1}{|P(x)|}\end{aligned}$$

This means that $\frac{\partial F}{\partial \theta}$ is bounded. The mean value theorem can then be used to show that F is Lipschitz continuous with respect to θ. Furthermore, F is continuous as a 2 variable function. Equation (3.5) can be re-written as

$$\frac{\partial \theta}{\partial x} = F(x,\theta)$$

Imposing the initial condition $\theta(a) = \gamma$, the existence and uniqueness theorem for first order ordinary differential equation gives a unique solution to (3.5). Equation (3.6) with the initial condition $r(a) = K$ can be solved as

follows

$$\int_a^x \frac{1}{r(s)}\frac{dr}{ds}\,ds = \int_a^x \frac{1}{2}\sin 2\theta\left[\frac{1}{P(s)} - Q(s)\right]\,ds$$
$$\log r(x) = \log r(a) + \frac{1}{2}\int_a^x \sin 2\theta\left[\frac{1}{P(s)} - Q(s)\right]\,ds$$
$$r(x) = K\exp\left(\frac{1}{2}\int_a^x \sin 2\theta\left[\frac{1}{P(s)} - Q(s)\right]\,ds\right)$$

We set $P(x) = p(x)$ and $Q(x) = \lambda w(x) - q(x)$ where $p \in C^1([a,b],\mathbb{R}), w, q \in C^0([a,b],\mathbb{R})$ with $p, w > 0$. In this case, equation (3.1) becomes (1.1). So we can associate the following Prüfer system with the Sturm-Liouville system (1.1,1.2) by substituting into (3.5,3.6)

$$\frac{d\theta}{dx} = (\lambda w(x) - q(x))\sin^2\theta + \frac{\cos^2\theta}{p(x)} \tag{3.10a}$$
$$\frac{dr}{dx} = \frac{1}{2}r\sin 2\theta\left[\frac{1}{p(x)} - \lambda w(x) + q(x)\right] \tag{3.10b}$$

From the definition of the Prüfer substitution and (1.2), for λ to be an eigenvalue, the solution, $\theta(x,\lambda)$ must satisfy the following initial conditions:

$$\theta(a,\lambda) = \arctan\left(-\frac{\beta_0}{\alpha_0 p(a)}\right) \tag{3.11a}$$
$$\theta(b,\lambda) = \arctan\left(-\frac{\beta_1}{\alpha_1 p(b)}\right) \tag{3.11b}$$

4 The Shooting method

In this section, we explain the Prüfer based shooting method to estimate eigenvalues of regular Sturm-Liouville problems with Dirichlet-Neumann boundary conditions. The general approach is to approximate $\theta(x, \lambda)$ for a given value of λ with one of the boundaries fixed. We then vary λ until the second condition is also (approximately) satisfied. It is similar to the approach used in [5].

We want to approximate the first eigenvalue for the following problem:

$$(p(x)y')' + \lambda w(x)y = 0 \tag{4.1}$$

on $[a, b]$ with the Dirichlet-Neumann boundary conditions

$$y(a) = 0 = y'(b) \tag{4.2}$$

Here we assume that p and w are both second order continuously differentiable and strictly positive.

We know from the previous section that the eigenvalues of the Sturm-Liouville problem can be found by solving the initial value problem (3.5,3.11a). We can simply solve that differential equation numerically for a fixed test eigen-

value. Since, the boundary Dirichlet-Neumann boundary conditions have $\alpha_1 = 0, \beta_1 = 1$ in (1.2) and $p(b) > 0$,the eigenvalues must satisfy

$$\begin{aligned}\theta(b, \lambda_n) &= \lim_{\alpha_1 \to 0^+} \arctan\left(\frac{-1}{\alpha_1 p(b)}\right) + n\pi \\ &= \left(n - \frac{1}{2}\right)\pi\end{aligned}$$

and

$$\theta(a, \lambda_n) = 0$$

So, it would make sense to stop varying λ_1 when we obtain $\theta(b, \lambda_1) = \frac{\pi}{2}$. However, it is unlikely that we will get the exact value of λ_1 and $\theta(b, \lambda_1)$ will not be exactly $\frac{\pi}{2}$. So we terminate the algorithm when $|\theta(b, \lambda_1) - \frac{\pi}{2}| < \epsilon$ for some $\epsilon > 0$. Picking a smaller value of ϵ will make the value of λ_1 more precise but will require solving the differential equation for θ more times. We need to now establish how we will pick the initial estimates for λ.

The following theorem proven by *Breuer and Gottlieb* [6] gives the *a priori* bounds on the eigenvalues for such Sturm-Liouville problems.

Theorem 4.1. *Let λ_1 be the smallest eigenvalue of the system (4.1, 4.2). Then,*

$$b_1 := \frac{\pi^2 k^2}{4\left[\int_a^b w(x)\,dx\right]^2} \leq \lambda_1 \leq \frac{\pi^2 K^2}{4\left[\int_a^b w(x)\,dx\right]^2} =: b_2$$

where

$$k^2 = \min_{x\in[a,b]} pw, K^2 = \max_{x\in[a,b]} pw.$$

So we can start with testing b_1 and b_2 as candidates for λ_1 then testing the midpoint. If the midpoint is not sufficiently close to λ_1, we will have to continue to vary our candidate λ_1. We then pick new bounds b_1, b_2 for λ_1 depending on which the old bounds is closer to the value of λ_1. How that is practically done is detailed in the algorithm here:

1. Compute the *a priori* bounds (b_1 and b_2) on λ_1.
2. Use the Runge-Kutta 4 method to approximate $\theta(b, b_1)$ and $\theta(b, b_2)$.
3. If $|\theta(b, b_1) - \frac{\pi}{2}| < \epsilon$ (or $|\theta(b, b_2) - \frac{\pi}{2}| < \epsilon$), then $\lambda_1 = b_1$ (or $\lambda_1 = b_2$, respectively). Otherwise, continue the algorithm.
4. Set $b_{mid} = \frac{b_1+b_2}{2}$ and approximate $\theta(b, b_{mid})$.
5. If $|\theta(b, b_{mid}) - \frac{\pi}{2}| < \epsilon$, $\lambda_1 = b_{mid}$. Otherwise, continue.
6. If $|\theta(b, b_1) - \frac{\pi}{2}| < |\theta(b, b_2) - \frac{\pi}{2}|$ then set $b_2 = b_{mid}, \theta(b_2) = \theta(b_{mid})$. Otherwise, set $b_1 = b_{mid}, \theta(b_1) = \theta(b_{mid})$. In either case, go back to step 2.

In the next section, we discuss how the algorithm is implemented as a python program.

5 Implementation

To implement the algorithm described in the previous section we use the general purpose programming language, Python. This language is useful for many applications including mathematics since we can use several packages implemented in the language and the language is simple to use.[7] One such package that we make use of is Sympy which implements a computer algebra system. This lets do many important tasks needed to implement the algorithm including taking integrals, derivatives and finding roots of functions defined symbolically. [8] The python source code for the project can be found on GitHub (https://github.com/Adam-Miles/Eigenvalue-Solver). Here, we describe the key functions and variables that make up the program. First, we have the following variables declare globally for easy modification. This allows us to define the Sturm-Liouville problem whose eigenvalue we want to find.

a,b

These variables represent the boundary points for the Sturm-Liouville problem. These can be set to any floating point number. Note that the program will take longer to run if the difference between a and b is large since the Runge-Kutta method will require more steps to stay accurate.

x

This is simply a Sympy symbol for x that can be used to define expressions for $p(x)$ and $q(x)$.

p,q

Sympy expressions to represent the functions $p(x)$ and $q(x)$ using the symbol **x**. See the Sympy documentation for full details on how to define expressions.

N_STEPS

The number of evaluation steps when solving (3.5) numerically.

ANGLE_TOL

The value of ϵ for which we exit when $|\theta(b, \lambda_1) - \frac{\pi}{2}| < \epsilon$.

BISECTION_MAX_ITERS

The maximum number of times we adjust our value of λ_1 before exiting.

SCAN_POINTS

The number of points used when verifying that there is a solution to $\theta(b, \lambda_1) = \frac{\pi}{2}$.

Now, we describe the functions created to implement the shooting method.

p_eval, w_eval

The lambdified functions for p and w. This is used to evaluate them numerically.

compute_bounds()

Here we compute the *a priori* bounds for the smallest eigenvalue using theorem. It does so by symbolically computing the critical points of pw so that we can obtain its global maximum and minimum on $[a, b]$. We also symbolically compute $\int_a^b p(x)\,dx$ in the calculation.

get_F(lam)

This returns a python function in terms of the value of the *lam* that represents the function in (3.9). The returned function takes parameters for x and θ as numbers.

runge_kutta4_theta(lam, n_steps=N_STEPS)

Here we solve equation (3.10b) numerically at b with **lam** set to the trial eigenvalue using the Runge-Kutta 4 method.

g(lam)

This evaluates the residual function $g(\lambda_1) := \theta(b, \lambda_1) - \frac{\pi}{2}$.

find_bracket()

This function verifies that the equation $g(\lambda_1) = 0$ has a solution in the interval given by the *a priori* bounds and extends the interval if it does not.

bisect_root(left, right, tol=ANGLE_TOL, max_iters =BISECTION_MAX_ITERS)

This implements the bisection root finding method. So, we vary λ_1 until a good approximation is found (ie. $|g(\lambda_1)| < \epsilon$).

compute_eigenvalue()

This function calls all the functions necessary to compute the eigenvalues and compare them with the reference values.

6 Examples

Here we text the code against some Sturm-Liouville problems in which the eigenvalues can be solved analytically or approximated as the solution to a transcendental equation. In these examples we take $\mathbf{N\_STEPS = 20000}$, $\mathbf{ANGLE\_TOL = 10^{-12}}$, $\mathbf{BISECTION\_MAX\_ITERS = 200}$ and $\mathbf{SCAN\_POINTS = 400}$.

6.1 Example 1

The first Sturm-Liouville problem that we test the shooting method against is the following.

$$y'' + \lambda y = 0 \tag{6.1a}$$

$$y(0) = y'(1) = 0 \tag{6.1b}$$

Since (6.1a) is a second order linear differential equation, we first solve the auxiliary equation,

$$r^2 + \lambda = 0$$

That means $r = \pm\sqrt{-\lambda}$. We must now break this problem into cases depending on the values of λ.

Case 1: $\lambda < 0$

Here we have two real values of r and the solution to (6.1a) has the form

$$y = c_1 e^{\sqrt{-\lambda}x} + c_2 e^{-\sqrt{-\lambda}x}$$

and its derivative is

$$y' = c_1\sqrt{-\lambda}e^{\sqrt{-\lambda}x} - c_2\sqrt{-\lambda}xe^{-\sqrt{-\lambda}x}$$

The boundary conditions imply,

$$y(0) = c_1 + c_2 = 0$$

and so $c_2 = -c_1$. Now,

$$y'(1) = c_1\sqrt{-\lambda}\left(e^{\sqrt{-\lambda}} + e^{-\sqrt{-\lambda}}\right) = 0$$

Since, this gives $c_1 = c_2 = 0$, if $\lambda < 0$, we only have the trivial solution. Thus, there are no eigenvalues with $\lambda < 0$.

Case 2: $\lambda = 0$

Our solutions are of the form

$$y = c_1 x + c_2$$

and the derivative is

$$y' = c_1$$

Applying the boundary conditions gives $y(0) = c_2 = 0$ and $y'(1) = c_1 = 0$. Given that there are no non-trivial solutions with $\lambda = 0$, it is not an eigenvalue.

Case 3: $\lambda > 0$

In this case, we have

$$y = c_1 \cos(\sqrt{\lambda}x) + c_2 \sin(\sqrt{\lambda}x)$$

and

$$y' = -c_1\sqrt{\lambda}\sin(\sqrt{\lambda}x) + c_2\sqrt{\lambda}\cos(\sqrt{\lambda}x)$$

Using the initial conditions, we obtain

$$y(0) = c_1 = 0$$

and

$$\begin{aligned} y'(1) = c_2\sqrt{\lambda}\cos(\sqrt{\lambda}) &= 0 \\ \cos(\sqrt{\lambda}) &= 0 \\ \sqrt{\lambda} &= \frac{(2n+1)\pi}{2}, \text{ for some } n \in \mathbb{N} \\ \lambda_n &= \frac{(2n+1)^2\pi^2}{4} \end{aligned}$$

Using that formula, an approximate value for λ_1 is 2.46740110027234. The Python code gives $\lambda_1 \approx 2.46740110027234$. This gives a forward error of 0. Of course this doesn't mean that the code gives a perfect value for λ_1 but on this example it is correct to 14 decimal places.

6.2 Example 2

We consider the problem

$$(x^3y')' + \lambda xy = 0 \tag{6.2a}$$

$$y(1) = 0 = y'(2) \tag{6.2b}$$

The expanding out (6.2a), we get

$$\begin{aligned} x^3y'' + 3x^2y' + \lambda xy &= 0 \\ x^2y'' + 3xy' + \lambda y &= 0 \end{aligned}$$

Now, we have a Cauchy-Euler equation with the auxiliary equation,

$$r(r-1)+3r+\lambda = r^2+2r+\lambda = 0$$

One can use the quadratic formula to get the solutions

$$r = -1 \pm \sqrt{1-\lambda}$$

Now we must consider cases for the solutions to (6.2a).

Case 1: $\lambda < 1$

The solution takes the form

$$y = c_1 x^{-1-\sqrt{1-\lambda}} + c_2 x^{-1+\sqrt{1-\lambda}}$$

and

$$y' = c_1(-1-\sqrt{1-\lambda})x^{-2-\sqrt{1-\lambda}} + c_2(-1+\sqrt{1-\lambda})x^{-2+\sqrt{1-\lambda}}$$

By (6.2b),

$$y(1) = 0 = c_1 + c_2$$

and,

$$\begin{aligned} 0 &= y'(2) \\ &= \frac{1}{4}c_1 \left[(-1-\sqrt{1-\lambda})2^{-\sqrt{1-\lambda}} + (1-\sqrt{1-\lambda})2^{\sqrt{1-\lambda}}\right] \end{aligned}$$

which has no solutions for $\lambda < 1$.

Case 2: $\lambda = 1$

Under this condition for λ,

$$\begin{aligned} y &= c_1 \frac{\log(x)}{x} + c_2 \frac{1}{x} \\ y' &= c_1 \frac{1-\log x}{x^2} - c_2 \frac{1}{x^2} \end{aligned}$$

The boundary conditions tell us that

$$\begin{aligned} y(1) &= c_2 = 0 \\ y'(2) &= \frac{1-\log 2}{4} c_1 = 0 \end{aligned}$$

So we again get a trivial solution to the equation and $\lambda = 1$ is not an eigenvalue.

Case 3: $\lambda > 1$

Our solutions are

$$y = c_1 \frac{\cos(\sqrt{\lambda - 1}\log x)}{x} + c_2 \frac{\sin(\sqrt{\lambda - 1}\log x)}{x}$$

which means that

$$\begin{aligned} y' = &- c_1 \frac{\sqrt{\lambda - 1}\sin(\sqrt{\lambda - 1}\log x) + \cos(\sqrt{\lambda - 1}\log x)}{x^2} \\ &+ c_2 \frac{\sqrt{\lambda - 1}\cos(\sqrt{\lambda - 1}\log x) - \sin(\sqrt{\lambda - 1}\log x)}{x^2} \end{aligned}$$

Applying the initial boundary conditions,

$$\begin{aligned} y(1) &= c_1 = 0 \\ y'(2) &= \frac{1}{4} c_2 \left[\sqrt{\lambda - 1}\cos(\log 2\sqrt{\lambda - 1}) - \sin(\log 2\sqrt{\lambda - 1})\right] = 0 \\ 0 &= \sqrt{\lambda - 1}\cos(\log 2\sqrt{\lambda - 1}) - \sin(\log 2\sqrt{\lambda - 1}) \end{aligned}$$

where we assume that y is a non-trivial solution so that $c_2 \neq 0$. Computing the smallest value of $\lambda > 1$ explicitly is not possible so we need to approximate it numerically. Using Wolfram alpha we see that $\lambda_1 \approx 2.8025510350225100$. The shooting method gives $\lambda_1 \approx 2.8025510350228080$ and hence its error is 2.980×10^{-13}.

6.3 Example 3

For the last test we solve

$$(e^x y')' + \lambda e^x y = 0 \tag{6.3a}$$

$$y(0) = y'(1) = 0 \tag{6.3b}$$

Solving (6.3a) is equivalent to solving the following second order linear differential equation with constant coefficients

$$y'' + y' + \lambda y = 0$$

which has the auxiliary equation

$$r^2 + r + \lambda = 0$$

with solutions

$$r = -\frac{1}{2} \pm \sqrt{\frac{1}{4} - \lambda}$$

Like in the previous example, we now consider cases for the solution depending on the value of λ.

Case 1: $\lambda < \frac{1}{4}$

With such values of λ, we get the solution

$$y = c_1 e^{\left(-\frac{1}{2}+\sqrt{\frac{1}{4}-\lambda}\right)x} + c_2 e^{\left(-\frac{1}{2}-\sqrt{\frac{1}{4}-\lambda}\right)x}$$

and its derivative is

$$y' = c_1 \left(-\frac{1}{2} + \sqrt{\frac{1}{4}-\lambda}\right) e^{\left(-\frac{1}{2}+\sqrt{\frac{1}{4}-\lambda}\right)x} + c_2 \left(-\frac{1}{2} - \sqrt{\frac{1}{4}-\lambda}\right) e^{\left(-\frac{1}{2}-\sqrt{\frac{1}{4}-\lambda}\right)x}$$

.

By (6.3b),

$$\begin{aligned}
y(0) &= c_1 + c_2 = 0 \\
y'(1) &= c_1 \left(-\frac{1}{2} + \sqrt{\frac{1}{4}-\lambda}\right) e^{-\frac{1}{2}+\sqrt{\frac{1}{4}-\lambda}} - c_1 \left(-\frac{1}{2} - \sqrt{\frac{1}{4}-\lambda}\right) e^{-\frac{1}{2}-\sqrt{\frac{1}{4}-\lambda}} \\
0 &= c_1 \left(-\frac{1}{2} + \sqrt{\frac{1}{4}-\lambda}\right) e^{-\frac{1}{2}+\sqrt{\frac{1}{4}-\lambda}} - c_1 \left(-\frac{1}{2} - \sqrt{\frac{1}{4}-\lambda}\right) e^{-\frac{1}{2}-\sqrt{\frac{1}{4}-\lambda}} \\
&= c_1 e^{-\frac{1}{2}} \left[\frac{1}{2}\left(e^{-\sqrt{\frac{1}{4}-\lambda}} - e^{\sqrt{\frac{1}{4}-\lambda}}\right) + \sqrt{\frac{1}{4}-\lambda}\left(e^{-\sqrt{\frac{1}{4}-\lambda}} + e^{\sqrt{\frac{1}{4}-\lambda}}\right)\right]
\end{aligned}$$

Case 2: $\lambda = \frac{1}{4}$

The solution for this case is

$$y = c_1 e^{-\frac{1}{2}x} + c_2 x e^{-\frac{1}{2}x}$$

now we have

$$y' = -\frac{1}{2}c_1 e^{-\frac{1}{2}x} + c_2\left(e^{-\frac{1}{2}x} - \frac{1}{2}xe^{-\frac{1}{2}x}\right)$$

Applying the associated boundary conditions,

$$y(0) = c_1 = 0$$
$$y'(1) = \frac{1}{2}e^{-\frac{1}{2}}c_2 = 0$$

So there are no eigenfunctions for $\lambda = \frac{1}{4}$ and it is not an eigenvalue.

Case 3: $\lambda > \frac{1}{4}$

Here, the solution takes the form

$$y = e^{-\frac{1}{2}x}\left[c_1 \sin\left(\sqrt{\lambda - \frac{1}{4}}x\right) + c_2 \cos\left(\sqrt{\lambda - \frac{1}{4}}x\right)\right]$$

Making the derivative

$$\begin{aligned} y' = &-\frac{1}{2}e^{-\frac{1}{2}x}\left[c_1 \sin\left(\sqrt{\lambda - \frac{1}{4}}x\right) + c_2 \cos\left(\sqrt{\lambda - \frac{1}{4}}x\right)\right] \\ &+ e^{-\frac{1}{2}x}\left[\sqrt{\lambda - \frac{1}{4}}c_1 \cos\left(\sqrt{\lambda - \frac{1}{4}}x\right) - \sqrt{\lambda - \frac{1}{4}}c_2 \sin\left(\sqrt{\lambda - \frac{1}{4}}x\right)\right] \end{aligned}$$

The boundary conditions give

$$y(0) = c_2 = 0$$
$$y'(1) = e^{-\frac{1}{2}} c_1 \left[-\frac{1}{2} \sin\left(\sqrt{\lambda - \frac{1}{4}}\right) + \sqrt{\lambda - \frac{1}{4}} \cos\left(\sqrt{\lambda - \frac{1}{4}}\right) \right] = 0$$

Like in example 2, we need to solve for λ numerically. Using wolfram alpha, we obtain the approximation $\lambda_1 \approx 1.6085328764616391$. The python code gives $\lambda_1 \approx 1.6085328764627076$ meaning that the error is 1.068×10^{-12}.

7 Conclusion

In this project, we studied the computation of eigenvalues for Sturm-Liouville problems. We began by establishing that there is indeed a smallest eigenvalue for regular Sturm-Liouville problems, so that our problem is theoretically solvable. The Prüfer substitution was introduced to help simplify the problem of finding the eigenvalue. We then introduced a shooting method to compute the smallest eigenvalue. Finally, we implemented the shooting method in Python to ultimately give us the solutions. Some tests were also provided to show that the algorithm provides accurate approximations of the smallest eigenvalue for some examples.

References

[1] Sam Melkonian. *Mathematical Methods and Boundary Value Problems*. Infinity Books, 2023.

[2] G Birkhoff and GC Rota. *Ordinary Differential Equations; 1989; 4 New York*.

[3] Anton Zettl. *Sturm-Liouville Theory*. 121. American Mathematical Soc., 2005.

[4] Robert Magnus. *Essential Ordinary Differential Equations*. Springer, 2023.

[5] Mohammad DM Abumosameh. "Python Code for Sturm-Liouville Eigenvalue Problems". MSc thesis. Carleton University, 2022.

[6] Shlomo Breuer and David Gottlieb. "Upper and lower bounds on eigenvalues of Sturm-Liouville systems". In: *Journal of Mathematical Analysis and Applications* 36.3 (1971), pp. 465–476.

[7] Al Sweigart. *Automate the boring stuff with Python*. No Starch Press, 2025.

[8] Aaron Meurer et al. "SymPy: Symbolic Computing in Python". In: *PeerJ Computer Science* 3 (2017), e103.